SOFTWARE PROJECT DEVELOPMENT / CAPSTONE DEFENSE

Kizuna Nihongo

An AI-powered, personalized platform for learning Japanese — JLPT N5 to N1.

AI Sensei | Personalized Roadmap | Proctored Exams

Kizuna Nihongo Team · React · Express · Supabase · FPT AI Factory · 2026

絆

絆

Landing page screenshot

assets/landing-hero.png

AGENDA

What we'll cover

I

Project Overview

Why we built Kizuna Nihongo — scope & technology stack

II

Reference Systems

Mazii & Bunpro — what we adopted, what we extended

III

Actors & Use Cases

4 actors · 75 use cases in 22 functional groups

IV

Database Design

Schema-per-domain — 10 module schemas, 96 tables

V

Code Package

12 packages across client, server, integrations & data

I

PART ONE

Project Overview

Why a Vietnamese learner needs one platform that owns the whole journey — and what we set out to build.

- Motivation
- Problem
- Solution
- Scope & Tech stack

Why we chose this problem

JLPT certification is the gate to jobs, scholarships and study abroad — yet the tooling around it is scattered.

DEMAND

Every Vietnamese learner climbs the same ladder — **N5 → N1** — and the certificate, not the study hours, is what employers and universities read.

FRAGMENTATION

One app for the dictionary, another for grammar drills, a third for mock tests, a video course somewhere else. **None of them share progress.**

COST OF FEEDBACK

The missing piece — correction on writing, speaking and kanji — comes from a tutor: expensive, hard to book, not on demand.

OPPORTUNITY

LLM APIs now make personalized guidance and instant correction affordable, so a small team can build what used to need a teaching staff.

One path for everyone doesn't work

Traditional Japanese study forces every learner down the same track — and leaves them without timely feedback.

01

Generic roadmap

Everyone follows a single fixed curriculum, ignoring their real level, pace and goal.

02

No instant feedback

Writing, speaking and kanji practice go uncorrected — mistakes stick.

03

Tutors don't scale

1-on-1 guidance is costly, hard to book and unavailable on demand.

04

Passive content

Static textbooks can't adapt to the exact points a learner struggles with.

A personalized, AI-assisted platform

Kizuna Nihongo turns a fixed curriculum into an adaptive experience — with AI woven through learning, authoring and assessment.

AI

Personalized roadmap

AI builds a step-by-step study plan from the learner's current level to their target JLPT.

AI

AI Sensei tutor

An agentic assistant that answers, reads the learner's own data, and acts only with confirmation.

AI

AI-assisted authoring

Teachers & admins generate readings, questions and audio in minutes, then review.

AI

Proctored assessment

Exams run with webcam face-detection and anti-cheat, plus auto & AI-assisted grading.

What we built & how

SCOPE · JLPT N5 → N1

- **Learn** — courses, lessons, vocabulary, kanji, grammar & a JP-VI dictionary
- **Practice** — flashcards, reading with furigana, writing & kanji handwriting
- **Assess** — quizzes, proctored mock exams, placement test & leaderboards
- **Personalize** — AI learning roadmap & AI Sensei chat
- **Author** — teachers & admins generate content with AI (readings, questions, audio)

TECHNOLOGY STACK

Frontend SPA
React 18 · Vite · Tailwind CSS · React Router

Backend API
Node.js · Express REST API (Bearer JWT)

SSR Web
EJS · Express (landing & auth callback)

Data · Auth · Storage
Supabase — PostgreSQL, GoTrue Auth, Storage

AI Services
FPT AI Factory (OpenAI-compatible)

Proctoring & Email
MediaPipe Vision · SMTP (Nodemailer)

II

PART TWO

Reference Systems

Two systems our learners already use every day — **Mazii** and **Bunpro**. We studied both, kept what works, and named the gaps we wanted to close.

Mazii · JP–VI dictionary

Bunpro · SRS grammar

Comparison matrix

Mazii — the dictionary every learner has

Mazii — screenshot

Replace with your image · assets/ref-mazii.png

WHAT IT IS

A Japanese–Vietnamese dictionary and study app: word lookup, kanji lookup, example sentences, saved word lists and JLPT practice tests.

WHAT WE ADOPTED

Its **layered lexical model** — entry → sense → kanji → example — became our `dictionary_module` (~17k entries, 28k senses), plus lookup-then-save-to-study-list as a core habit.

GAPS WE CLOSE

No structured course path, no teacher-authored lessons, no adaptive study plan and no proctored exam — it is a reference tool, not a curriculum.

Bunpro — spaced repetition, done well

WHAT IT IS

A grammar-first study platform where every grammar point and word is tagged by JLPT level and drilled through a spaced-repetition review queue.

WHAT WE ADOPTED

JLPT-level tagging on every item and **per-item progress** — the shape of our `language_module` and `flashcard_module` (folders → sets → cards → progress).

GAPS WE CLOSE

English-only UI, grammar-centred (little listening / speaking / writing), no live teachers and no full three-section JLPT simulation.

What we kept, what we added

Capability	Mazii	Bunpro	Kizuna Nihongo
JP–VI dictionary	✓ core product	—	✓ ~17k entries built in
Spaced repetition / flashcards	~ saved word lists	✓ core product	✓ folders → sets → cards + AI-generated tests
Structured courses (unit → lesson)	—	—	✓ 3-tier course · unit · lesson with progress
Teacher-authored content	—	—	✓ apply → author with AI → revenue share
Personalized AI roadmap	—	~ fixed level order	✓ current level → target, regenerable
Proctored JLPT mock exam	~ practice tests	—	✓ 3 sections, timers, webcam & anti-cheat
Monetization model	ads · premium	subscription	subscription + per-course + teacher payout

Our position: keep the lexical depth of Mazii and the review discipline of Bunpro, then wrap them in a curriculum, a teacher economy and a real exam simulator.

III

PART THREE

Actors & Use Cases

Who the system serves, and the **75 use cases** that make up the product — grouped into 22 functional areas.

Guest · Student · Teacher · Admin

Actors & stakeholders

Guest

Browses public content, registers & signs in.

Student

Learns, practices, takes exams & uses AI features.

Teacher

Authors content & question banks, grades exams.

Admin

Manages users, content, monetization & the system.

External · AI Service (FPT AI Factory)

External · Payment Gateway

External · SMTP Email

Roles (**student** / **teacher** / **admin**) are carried in the Supabase Auth JWT **user_metadata.role** and drive every route guard.

75 use cases, 22 functional groups

Use Case Diagram — system overview

Replace with your image · assets/uc-overview.png

UC-01 … 08 — **Authentication & Profile**
Guest / User · register, OTP, login, password, profile

UC-09 … 21 — **Learning & reference**
Student · courses, vocab/kanji/grammar, flashcards, dictionary

UC-22 … 30 — **Skill practice**
Student · reading, listening & speaking, writing

UC-31 … 37 — **Assess, AI & account**
Student · mock exams, AI Sensei, progress, subscription

UC-38 … 63 — **Content authoring**
Teacher & Admin · courses, posts, question bank, exams

UC-64 … 75 — **Operations**
Admin · users, payments, revenue sharing, analytics

Use Case · Authentication

ACTOR · GUEST / USER

- **Register** with email & 6-digit **OTP** verification
- **Log in** — email/password or **Google OAuth**
- **Forgot / reset password** via OTP (neutral response)
- **Change password** — re-authentication required
- **Role-based routing** after login (BR-05)

Use Case Diagram — Authentication

Replace with your image · assets/uc-auth.png

Use Case · Student

ACTOR · STUDENT

- **Learn** — enroll in courses, study lessons, vocab, kanji & grammar
- **Practice** — flashcards (AI-define / AI test), reading, writing, kanji
- **Assess** — quizzes, proctored mock exams, placement test, leaderboard
- **Personalize** — AI learning roadmap & AI Sensei chat
- **Account** — profile, subscription & upgrade

Use Case · Teacher

ACTOR · TEACHER

- **Apply** to become a teacher — **AI screening** of the application
- **Author content with AI** — reading passages (4-step pipeline) & listening
- **Question bank** — manual & AI-generated items in a private bank
- **Exams** — create exams & review attempts (AI grading assist)
- **Earnings** — monthly revenue share based on content usage

Use Case · Admin

ACTOR · ADMIN

- **User management** — roles, accounts & audit logs
- **Content** — course builder, vocab/kanji/grammar & approvals
- **Assessment** — question bank, mock exams (AI + TTS), placement config
- **Monetization** — subscriptions, payments & teacher revenue pool
- **Operations** — system health (App/DB/AI) & teacher-application review

IV

PART FOUR

Database Design

One PostgreSQL database, split into **10 domain schemas** — 96 tables, 138 relationships, RLS on every table.

Schema-per-domain | 96 tables | Cross-schema FKs | Triggers & RPC

Schema-per-domain, by design

10
domain schemas (+ `public` for system objects)

96
tables, all with Row Level Security enabled

138
foreign-key relationships, including cross-schema

01

One schema = one domain

Every table serving the same feature area lives together, named with the **_module** suffix.

02

Money is isolated

All financial tables sit in **billing_module**, granted to **service_role** only — easy to audit and lock down.

03

public is system-only

Sessions, settings, RPC functions and compatibility views — **no business tables**.

04

Two-layer authorization

RLS on every table; the backend uses **service_role**; the client never queries the database directly.

05

Cross-schema keys

Real foreign keys across domains; multi-schema joins are composed in the backend layer.

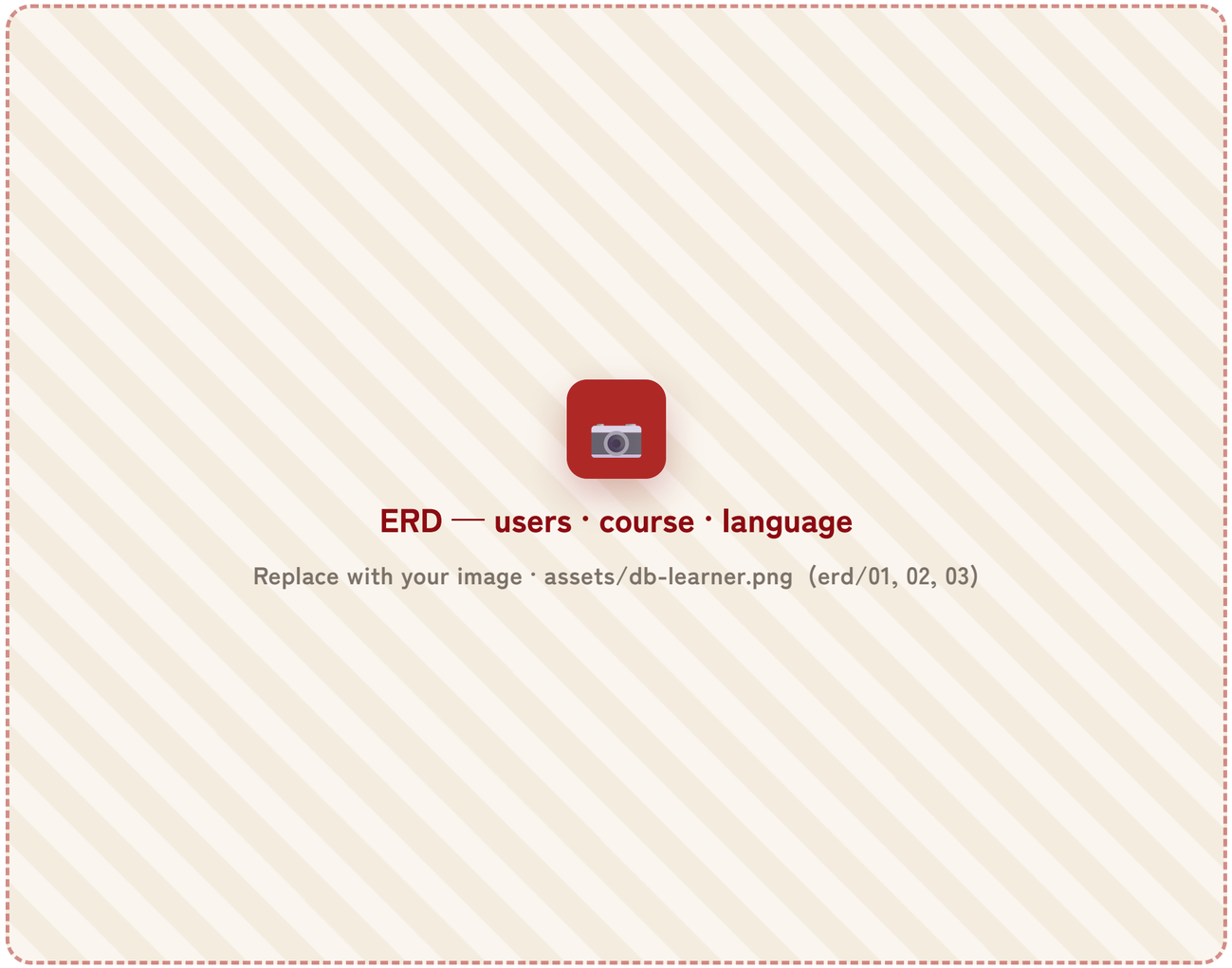
The whole database, one picture

ERD — all 96 tables grouped by schema

Replace with your image · assets/db-overview.png (export docs/erd/00-overview.dbml on dbdiagram.io)

users_module 6 tables · identity	**course_module** 11 · courses	**language_module** 17 · corpus	**practice_module** 12 · 4 skills	**flashcard_module** 6 · SRS	**exam_module** 9 · quizzes
jlpt_module 8 · mock exams	**dictionary_module** 5 · JP–VI	**billing_module** 11 · money	**ai_module** 9 · AI & paths	**public** 2 + views · system	**auth** Supabase Auth

Who learns, and what they learn



users_module 6 tables

Identity root is **auth.users**; the **handle_new_user** trigger provisions **users**, **student_profiles** and the AI dashboard. **teacher_applications** stores the AI verdict plus the admin decision.

course_module 11 tables

Three tiers — **courses → units → lessons** — with **course_enrollments**, **lesson_progress** and **course_reviews**; triggers cache rating and enrollment count.

language_module 17 tables

The shared corpus every feature reads: **vocabulary**, **kanji**, **grammar_points**, **grammar_patterns**, **jlpt_levels**, topics — plus study-list posts written by teachers and admins.

Practising and being tested

ERD — practice · flashcard · exam · jlpt

Replace with your image · assets/db-practice-exam.png (erd/04, 05, 06, 07)

practice_module 12 tables

All four skills in one domain: reading (`articles`, `reading_sets → passages → questions → options`), listening, `writing_submissions` and `pronunciation_assessments`.

flashcard_module 6 tables

Folders ↔ sets many-to-many, `flashcards`, `flashcard_progress` keyed by (student, card), and one AI-generated test per set.

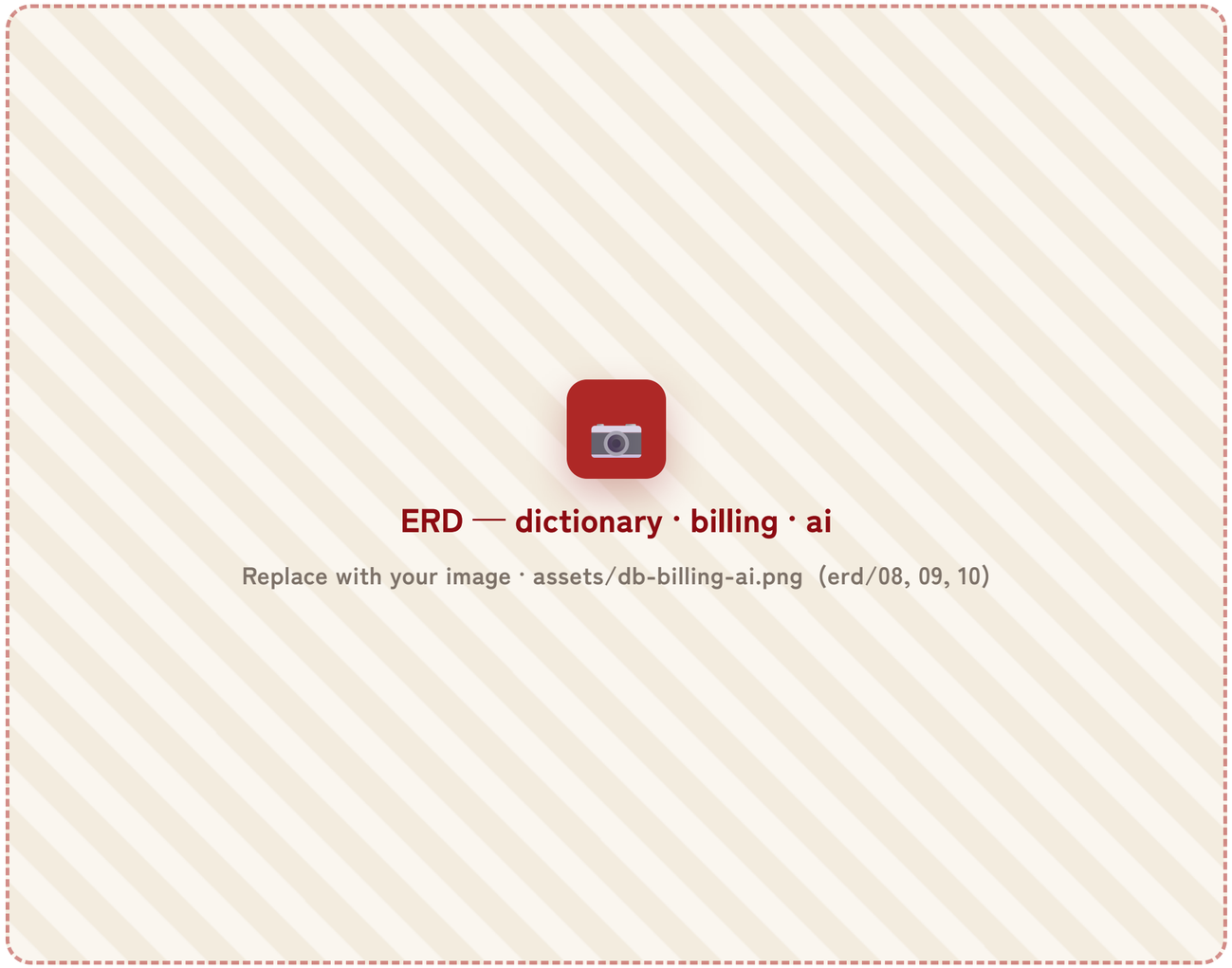
exam_module 9 tables

Course quizzes with proctoring flags; `quiz_questions` freeze a `passage_snapshot`; `quiz_lockouts` count fullscreen violations; separate admin and teacher question banks.

jlpt_module 8 tables

The real exam shape: `mock_exams → sections` (own timer) → `question_groups` (21 mondai types) → `questions`, plus attempts and a reusable JLPT bank.

Reference data, revenue & personalization



dictionary_module 5 tables

Imported at scale — ~17k entries, 28k senses, 10k kanji, 10k examples. Vietnamese-meaning lookup runs through the `search_dict_by_meaning` RPC (unaccent + ranking).

billing_module 11 tables

Two SePay flows told apart by transfer code — `PREM…` for subscriptions, `COURSE…` for courses — matched idempotently against bank transactions. Quotas live in `feature_entitlements`; teacher payouts are computed from `content_usage_events`.

ai_module 9 tables

AI Sensei conversations, `learning_paths` + steps with a polymorphic `resource_type` (course / study list / mock exam), `student_dashboards` and notifications.

How the domains stay connected

Mechanism	What it guarantees
handle_new_user	A signup in **auth.users** provisions **users**, **student_profiles** and **ai_module.student_dashboards** in one `SECURITY DEFINER` trigger.
trg_payments_enroll	A completed row in **billing_module.payments** inserts the matching **course_module.course_enrollments** — idempotently.
lesson_* join tables	Lessons reference the shared corpus in **language_module** instead of copying vocabulary, kanji or grammar.
vip_revenue() · teacher_uses()	Monthly RPCs split the premium revenue pool into **teacher_payouts** in proportion to content usage.
Polymorphic links	**study_list_items.item_id** and **learning_path_steps.resource_id** have no hard FK — validated in the backend.
RLS + service_role	RLS on 100% of business tables; **billing_module** is granted to **service_role** only; **public** compat views keep older code working.

Cross-schema diagram *(optional)*

Replace with your image · assets/db-crossschema.png

V

PART FIVE

Code Package

How the source is organised — **12 packages** across the client, the server, external integrations and the database.

- Layered architecture
- Client packages
- Server packages
- Component diagram

A layered architecture

Presentation
CLIENT

- **React SPA** — student · teacher · admin
- **EJS (SSR)** — landing · auth callback
- **MediaPipe** — in-browser face detection

▼ HTTPS · /api/* with Bearer JWT ▼

Business Logic
SERVER

- **Express REST API** — ~24 route groups under /api/*
- **Route guards** — requireAuth · requireRole
- **Controllers** — domain logic
- **Metrics** — traffic middleware

▼ service-role client ▼

Data & Auth
SUPABASE

- **PostgreSQL** — 10 module schemas
- **GoTrue Auth** — JWT · Google OAuth
- **Storage** — private buckets · signed URLs

External
SERVICES

- **FPT AI Factory** — chat · generation · grading
- **SMTP** — OTP email
- **Payment Gateway** — subscriptions

Client packages 1 – 5

1 `frontend/src/pages`

React 18 + Vite screens split by role — **public · student · teacher · admin**. Routing in `App.jsx` is guarded by `ProtectedRoute` / `TeacherRoute` / `AdminRoute` and wrapped in role layouts.

2 `frontend/src/components`

Design-system primitives in `ui/` (`Button`, `Modal`, `DataTable`, `FuriganaText`) plus feature widgets — dictionary, flashcards, mock exam, receipt — and route-guard wrappers.

3 `frontend/src/contexts`

App-wide providers: `AuthContext` (Supabase session + role), `LangContext` (vi / ja), `ConfirmContext`, `PageContext`.

4 `frontend/src/lib`

Client services: `api.js` (one axios instance whose interceptor attaches the Supabase Bearer JWT), `supabase.js` (PKCE client), `useProctoring` + `faceDetector` for exam monitoring.

5 `routes/` + `views/` — **the EJS server**

Express 5 server-rendered site with Postgres session auth (`connect-pg-simple`); routes map to controllers and render EJS views. Scope: the public landing page and the Supabase OAuth callback.

Server & data packages 6 – 12

6 `backend/routes/api`

Thin Express 4 routers — **~24 groups** under `/api/*`. They only wire middleware to controller methods; no business logic.

7 `backend/controllers`

One file per domain, pattern `exports.action = async (req,res)`. Read/write Postgres via `supabaseAdmin`, paginate, return `{data}` / `{error}`.

8 `backend/services`

Reusable business logic: `courseAccess`, `subscriptionService`, `quotaService`, payment matching & revenue pool, `examGuard`, `jaTokenizer` / `furiganaService`.

9 **Payment gateway — SePay**

`sepayClient.js` + webhook `POST /api/webhooks/sepay`: match a transfer by amount & code, grant access, record revenue, email the receipt.

10 `backend/config`

External clients: `supabase.js` (anon + service-role), `ai.js` (FPT AI Factory), `mailer.js` (SMTP), `audio.js` / `youtube.js`.

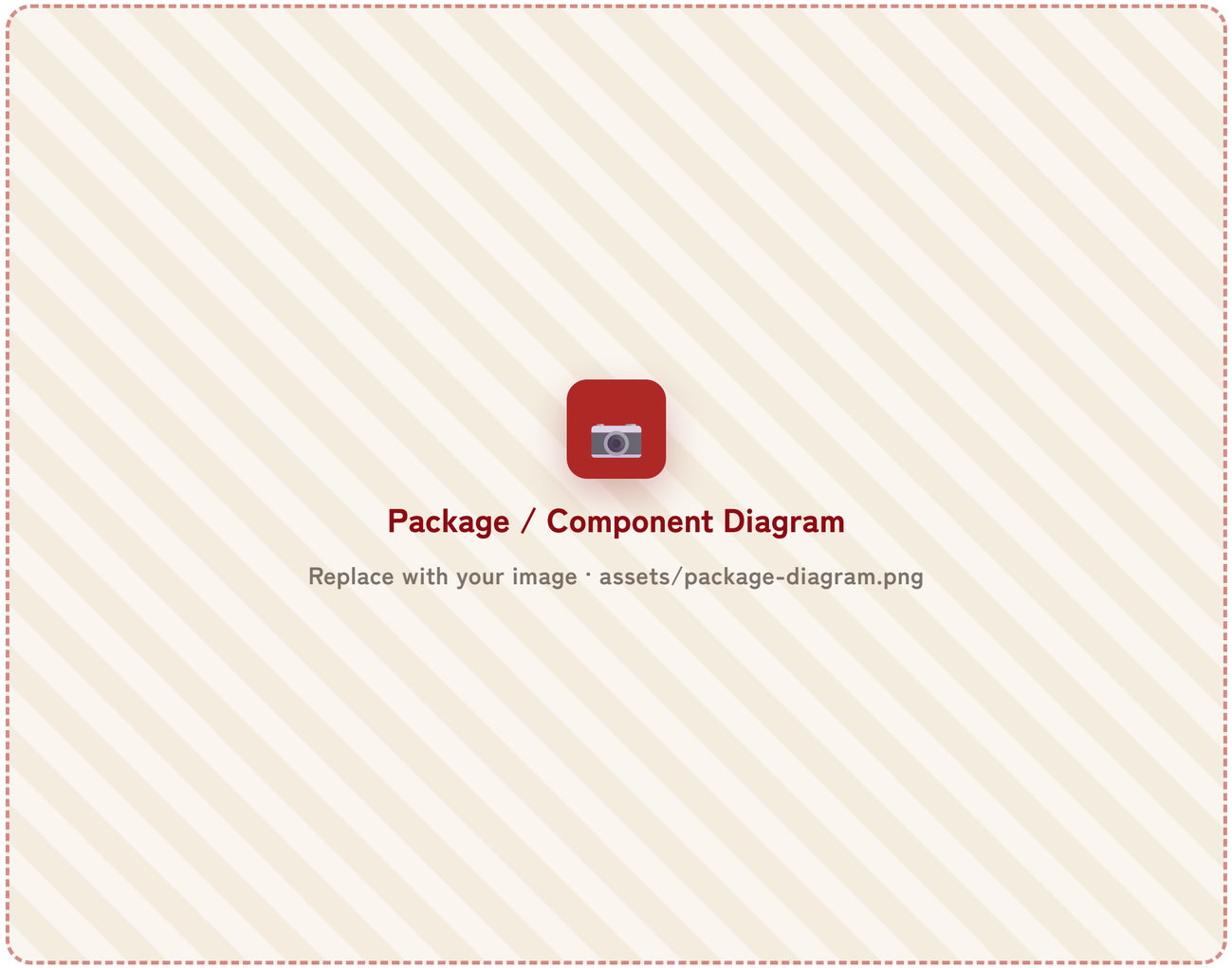
11 `backend/middleware`

Cross-cutting: `auth.js` (`requireAuth` / `requireTeacher` / `requireAdmin` / `optionalAuth`), `checkQuota.js`, `metrics.js`.

12 `database/` — **PostgreSQL schema for Supabase**

Idempotent `schema.sql` plus incremental `backend/migrations/NNN_*.sql`. Organised into the 10 feature schemas, with a `public` compatibility layer of views and `INSTEAD OF` triggers, and the `handle_new_user` provisioning trigger.

Packages, side by side



RULES WE HOLD TO

- **Routers stay thin** — they attach middleware and map to a controller, nothing else
- **One controller per domain** — request handling and Supabase queries live together
- **Shared logic becomes a service** — anything two controllers need moves into **backend/services**
- **External access is centralised** — every third-party client is created once in **backend/config**
- **The client never touches the database** — it goes through **/api/*** with a Bearer JWT

絆

Thank you

Kizuna Nihongo — connecting learners to Japanese, with AI.

Questions & Answers

Kizuna Nihongo Team · 2026